

CSE310: PROGRAMMING IN JAVA — MASTER NOTES FOR 100% EXAM PREPARATION

For LPU End Term + NeoColab Coding + Viva + MCQs

HOW TO USE THESE NOTES

1. Study every unit in order.
2. Type every code yourself.
3. Revise important points before exam.
4. Practice output questions.
5. Memorize syntax patterns.
6. Solve coding questions without seeing answers.
7. Revise viva questions daily.

If you complete these notes properly, you will be fully prepared for:

- MCQs
 - Viva
 - Written exam
 - NeoColab coding test
 - Practical file
 - Lab exam
-

UNIT I — INTRODUCTION TO JAVA

1. HISTORY OF JAVA

Java was developed by Sun Microsystems. James Gosling is called the father of Java.

Features:

- Platform Independent
- Object Oriented
- Secure
- Robust
- Multithreaded
- Portable
- High Performance

- Distributed
-

2. JDK vs JRE vs JVM

Term	Meaning
JVM	Java Virtual Machine runs bytecode
JRE	JVM + Libraries
JDK	JRE + Development tools

Flow: Java Source Code -> Compiler -> Bytecode -> JVM -> Output

3. BASIC JAVA PROGRAM

```
class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

IMPORTANT:

- main() is entry point.
 - String[] args stores command-line arguments.
 - System.out.println() prints with newline.
-

4. COMMAND LINE ARGUMENTS

```
class Cmd {  
    public static void main(String[] args) {  
        System.out.println(args[0]);  
        System.out.println(args[1]);  
    }  
}
```

Run:

```
java Cmd Aquib 21
```

Output:

```
Aquib  
21
```

5. PRIMITIVE DATA TYPES

Data Type	Size
byte	1 byte
short	2 bytes
int	4 bytes
long	8 bytes
float	4 bytes
double	8 bytes
char	2 bytes
boolean	true/false

Example:

```
int a = 10;  
double b = 5.5;  
char c = 'A';  
boolean flag = true;
```

6. TYPE CASTING

Implicit

```
int a = 10;  
double b = a;
```

Explicit

```
double x = 10.5;  
int y = (int)x;
```

7. KEYWORDS

Examples:

- class
- public
- static
- void
- int
- if
- else
- switch
- return
- try
- catch

Java keywords cannot be used as variable names.

8. VARIABLES

```
int age = 21;  
String name = "Aquib";
```

Types:

- Local variable
- Instance variable
- Static variable

9. ACCESS MODIFIERS

Modifier	Same Class	Same Package	Subclass	Other Package
private	YES	NO	NO	NO
default	YES	YES	NO	NO
protected	YES	YES	YES	NO
public	YES	YES	YES	YES

10. STATIC KEYWORD

```
class Test {  
    static int count = 0;  
  
    Test() {  
        count++;  
    }  
  
    static void display() {  
        System.out.println(count);  
    }  
}
```

Static belongs to class.

11. WRAPPER CLASSES

Primitive	Wrapper
int	Integer
char	Character
double	Double
boolean	Boolean

```
Integer x = Integer.valueOf(10);  
int y = x.intValue();
```

Autoboxing:

```
Integer a = 10;
```

Unboxing:

```
int b = a;
```

12. OPERATORS

Arithmetic

```
+ - * / %
```

Relational

```
== != > < >= <=
```

Logical

```
&& || !
```

Bitwise

```
& | ^ << >>
```

Ternary

```
int max = (a > b) ? a : b;
```

13. IF ELSE

```
if(a > b) {  
    System.out.println(a);  
} else {  
    System.out.println(b);  
}
```

14. SWITCH

```
int day = 2;  
  
switch(day) {  
    case 1:  
        System.out.println("Mon");  
        break;  
  
    case 2:  
        System.out.println("Tue");  
        break;  
  
    default:  
        System.out.println("Invalid");  
}
```

UNIT II — LOOPS, ARRAYS, OOP, STRINGS

1. FOR LOOP

```
for(int i=1;i<=5;i++) {  
    System.out.println(i);  
}
```

2. WHILE LOOP

```
int i=1;
while(i<=5) {
    System.out.println(i);
    i++;
}
```

3. DO WHILE LOOP

```
int i=1;
do {
    System.out.println(i);
    i++;
} while(i<=5);
```

4. FOR EACH LOOP

```
int arr[] = {1,2,3};

for(int x : arr) {
    System.out.println(x);
}
```

5. ARRAYS

```
int arr[] = new int[5];
```

Initialization:

```
int arr[] = {10,20,30};
```

Traversal:


```
for(int i=0;i<arr.length;i++) {  
    System.out.println(arr[i]);  
}
```

6. 2D ARRAY

```
int a[][] = {  
    {1,2},  
    {3,4}  
};  
  
for(int i=0;i<2;i++) {  
    for(int j=0;j<2;j++) {  
        System.out.print(a[i][j] + " ");  
    }  
    System.out.println();  
}
```

7. VARARGS

```
class Test {  
    static int sum(int... x) {  
        int s=0;  
  
        for(int i : x)  
            s += i;  
  
        return s;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(sum(1,2,3));  
    }  
}
```

8. ENUM

```
enum Day {  
    MON, TUE, WED  
}  
  
class Demo {  
    public static void main(String[] args) {  
        Day d = Day.MON;  
        System.out.println(d);  
    }  
}
```

9. CLASS AND OBJECT

```
class Student {  
    int id;  
    String name;  
  
    void display() {  
        System.out.println(id + " " + name);  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
  
        s1.id = 1;  
        s1.name = "Aquib";  
  
        s1.display();  
    }  
}
```

10. CONSTRUCTOR

```
class Student {  
    int id;
```

```
Student(int x) {  
    id = x;  
}  
}
```

Constructor:

- Same name as class
- No return type
- Automatically called

11. CONSTRUCTOR OVERLOADING

```
class Test {  
    Test() {  
        System.out.println("Default");  
    }  
  
    Test(int a) {  
        System.out.println(a);  
    }  
}
```

12. METHOD OVERLOADING

```
class Add {  
    int sum(int a,int b) {  
        return a+b;  
    }  
  
    double sum(double a,double b) {  
        return a+b;  
    }  
}
```

13. THIS KEYWORD

```
class Student {  
    int id;  
  
    Student(int id) {  
        this.id = id;  
    }  
}
```

Uses:

- Refer current object
- Call constructor
- Pass object

14. INITIALIZER BLOCK

```
class Test {  
    {  
        System.out.println("Initializer block");  
    }  
  
    Test() {  
        System.out.println("Constructor");  
    }  
}
```

15. STRING CLASS

```
String s = "Hello";
```

Important methods:

```
s.length();  
s.charAt(0);  
s.toUpperCase();  
s.toLowerCase();
```

```
s.equals("Hello");  
s.equalsIgnoreCase("hello");  
s.contains("ell");  
s.substring(1,4);  
s.replace('l','x');
```

16. STRINGBUILDER

Mutable string.

```
StringBuilder sb = new StringBuilder("Hello");  
  
sb.append(" World");  
sb.insert(0,"Hi ");  
sb.delete(0,3);  
sb.reverse();  
  
System.out.println(sb);
```

Difference:

- String immutable
- StringBuilder mutable

UNIT III — INHERITANCE, POLYMORPHISM, INTERFACE

1. INHERITANCE

```
class Animal {  
    void sound() {  
        System.out.println("Animal sound");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Bark");  
    }  
}
```

```
}  
}
```

Types:

- Single
- Multilevel
- Hierarchical

Java does not support multiple inheritance using classes.

2. METHOD OVERRIDING

```
class A {  
    void show() {  
        System.out.println("A");  
    }  
}  
  
class B extends A {  
    void show() {  
        System.out.println("B");  
    }  
}
```

Rules:

- Same method name
 - Same parameters
 - Runtime polymorphism
-

3. SUPER KEYWORD

```
class A {  
    int x = 10;  
}  
  
class B extends A {  
    int x = 20;  
  
    void display() {
```

```
        System.out.println(super.x);  
        System.out.println(x);  
    }  
}
```

4. FINAL KEYWORD

Final Variable

```
final int x = 10;
```

Final Method

Cannot override.

Final Class

Cannot inherit.

```
final class A {  
}
```

5. OBJECT CLASS

Every class inherits Object class.

Important methods:

- toString()
- equals()
- hashCode()

6. toString()

```
class Student {  
    int id;  
  
    Student(int id) {  
        this.id = id;  
    }  
  
    public String toString() {  
        return "ID=" + id;  
    }  
}
```

7. equals()

```
class Student {  
    int id;  
  
    Student(int id) {  
        this.id = id;  
    }  
  
    public boolean equals(Object obj) {  
        Student s = (Student)obj;  
        return this.id == s.id;  
    }  
}
```

8. instanceof

```
if(obj instanceof String) {  
    System.out.println("Yes");  
}
```

9. ABSTRACT CLASS

```
abstract class Shape {  
    abstract void draw();  
}  
  
class Circle extends Shape {  
    void draw() {  
        System.out.println("Circle");  
    }  
}
```

Rules:

- Cannot create object
- Can contain abstract + normal methods

10. INTERFACE

```
interface Animal {  
    void sound();  
}  
  
class Dog implements Animal {  
    public void sound() {  
        System.out.println("Bark");  
    }  
}
```

Features:

- Multiple inheritance
- Methods are public abstract by default
- Variables are public static final

11. DEFAULT METHOD

```
interface A {  
    default void show() {
```

```
        System.out.println("Default");
    }
}
```

12. STATIC METHOD IN INTERFACE

```
interface A {
    static void display() {
        System.out.println("Static");
    }
}
```

Call:

```
A.display();
```

UNIT IV — NESTED CLASS, LAMBDA, EXCEPTION

1. STATIC NESTED CLASS

```
class Outer {
    static class Inner {
        void show() {
            System.out.println("Inner");
        }
    }
}
```

2. INNER CLASS

```
class Outer {
    class Inner {
        void show() {
            System.out.println("Inner");
        }
    }
}
```

```
}  
}
```

3. ANONYMOUS CLASS

```
abstract class A {  
    abstract void show();  
}  
  
class Test {  
    public static void main(String[] args) {  
        A obj = new A() {  
            void show() {  
                System.out.println("Anonymous");  
            }  
        };  
  
        obj.show();  
    }  
}
```

4. FUNCTIONAL INTERFACE

```
@FunctionalInterface  
interface A {  
    void show();  
}
```

Only one abstract method.

5. LAMBDA EXPRESSION

```
interface A {  
    void display();  
}  
  
class Test {
```

```
public static void main(String[] args) {  
  
    A obj = () -> {  
        System.out.println("Hello");  
    };  
  
    obj.display();  
}  
}
```

6. DATE CLASS

```
import java.util.Date;  
  
class Test {  
    public static void main(String[] args) {  
        Date d = new Date();  
        System.out.println(d);  
    }  
}
```

7. LOCAL DATE

```
import java.time.LocalDate;  
  
class Test {  
    public static void main(String[] args) {  
        LocalDate d = LocalDate.now();  
        System.out.println(d);  
    }  
}
```

8. EXCEPTION HANDLING

try catch

```
class Test {  
    public static void main(String[] args) {  
        try {  
            int a = 10/0;  
        }  
        catch(Exception e) {  
            System.out.println(e);  
        }  
    }  
}
```

9. FINALLY

```
try {  
    int x = 10/0;  
}  
catch(Exception e) {  
    System.out.println(e);  
}  
finally {  
    System.out.println("Always executes");  
}
```

10. THROW

```
throw new ArithmeticException("Error");
```

11. THROWS

```
class Test {  
    static void check() throws Exception {
```

```
        throw new Exception("Error");
    }
}
```

12. MULTI CATCH

```
try {
    int a = 10/0;
}
catch(ArithmeticException | ArrayIndexOutOfBoundsException e) {
    System.out.println(e);
}
```

13. TRY WITH RESOURCES

```
import java.io.*;

class Test {
    public static void main(String[] args) {

        try(FileReader fr = new FileReader("a.txt")) {
            System.out.println("Opened");
        }
        catch(Exception e) {
            System.out.println(e);
        }
    }
}
```

14. CUSTOM EXCEPTION

```
class MyException extends Exception {
    MyException(String msg) {
        super(msg);
    }
}
```

15. ASSERTION

```
int age = -1;
assert age > 0 : "Invalid age";
```

Enable assertion:

```
java -ea Test
```

UNIT V — IO, GENERICS, THREADS

1. INPUT USING SCANNER

```
import java.util.*;

class Test {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int x = sc.nextInt();
        String s = sc.next();

        System.out.println(x);
        System.out.println(s);
    }
}
```

2. FILE WRITING

```
import java.io.*;

class Test {
    public static void main(String[] args) throws Exception {

        FileWriter fw = new FileWriter("abc.txt");
```

```
        fw.write("Hello");

        fw.close();
    }
}
```

3. FILE READING

```
import java.io.*;

class Test {
    public static void main(String[] args) throws Exception {

        FileReader fr = new FileReader("abc.txt");

        int i;

        while((i = fr.read()) != -1) {
            System.out.print((char)i);
        }

        fr.close();
    }
}
```

4. BUFFEREDREADER

```
import java.io.*;

class Test {
    public static void main(String[] args) throws Exception {

        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));

        String s = br.readLine();

        System.out.println(s);
    }
}
```



```
}  
}
```

5. SERIALIZATION

```
import java.io.*;  
  
class Student implements Serializable {  
    int id = 1;  
}  
  
class Test {  
    public static void main(String[] args) throws Exception {  
  
        Student s = new Student();  
  
        FileOutputStream fos = new FileOutputStream("a.txt");  
        ObjectOutputStream oos = new ObjectOutputStream(fos);  
  
        oos.writeObject(s);  
  
        oos.close();  
    }  
}
```

6. GENERICS

```
class Box<T> {  
    T value;  
  
    void set(T value) {  
        this.value = value;  
    }  
  
    T get() {  
        return value;  
    }  
}
```

7. GENERIC ARRAYLIST

```
ArrayList<Integer> list = new ArrayList<>();
```

Diamond operator:

```
ArrayList<String> list = new ArrayList<>();
```

8. BOUNDED TYPES

```
class Test<T extends Number> {  
}
```

9. WILDCARDS

```
List<?> list;
```

10. THREAD USING THREAD CLASS

```
class MyThread extends Thread {  
    public void run() {  
        for(int i=1;i<=5;i++) {  
            System.out.println(i);  
        }  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        MyThread t = new MyThread();  
        t.start();  
    }  
}
```

11. THREAD USING RUNNABLE

```
class MyThread implements Runnable {  
    public void run() {  
        System.out.println("Running");  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        Thread t = new Thread(new MyThread());  
        t.start();  
    }  
}
```

12. THREAD USING LAMBDA

```
class Test {  
    public static void main(String[] args) {  
  
        Runnable r = () -> {  
            for(int i=1;i<=5;i++) {  
                System.out.println(i);  
            }  
        };  
  
        Thread t = new Thread(r);  
        t.start();  
    }  
}
```

13. THREAD PRIORITY

```
Thread t = new Thread();  
  
t.setPriority(Thread.MAX_PRIORITY);
```

Values:

- MIN_PRIORITY = 1
 - NORM_PRIORITY = 5
 - MAX_PRIORITY = 10
-

14. SYNCHRONIZATION

```
class Table {
    synchronized void print(int n) {
        for(int i=1;i<=5;i++) {
            System.out.println(n*i);
        }
    }
}
```

15. INTER THREAD COMMUNICATION

Methods:

- wait()
 - notify()
 - notifyAll()
-

UNIT VI — COLLECTIONS + JDBC

1. ARRAYLIST

```
import java.util.*;

class Test {
    public static void main(String[] args) {

        ArrayList<String> list = new ArrayList<>();

        list.add("A");
        list.add("B");
        list.add(1,"X");
    }
}
```

```
        System.out.println(list);

        list.remove("A");

        System.out.println(list.get(0));

        for(String s : list) {
            System.out.println(s);
        }
    }
}
```

Important methods:

- add()
- remove()
- get()
- set()
- contains()
- size()
- clear()

2. LINKEDLIST

```
LinkedList<Integer> list = new LinkedList<>();
```

Methods:

- addFirst()
- addLast()
- removeFirst()
- removeLast()

3. HASHSET

```
HashSet<Integer> set = new HashSet<>();

set.add(10);
set.add(20);
set.add(10);
```

```
System.out.println(set);
```

Features:

- No duplicates
- Unordered

4. TREESSET

```
TreeSet<Integer> set = new TreeSet<>();

set.add(30);
set.add(10);
set.add(20);

System.out.println(set);
```

Features:

- Sorted
- No duplicates

5. COMPARABLE

```
import java.util.*;

class Student implements Comparable<Student> {
    int id;
    String name;

    Student(int id,String name) {
        this.id = id;
        this.name = name;
    }

    public int compareTo(Student s) {
        return this.name.compareTo(s.name);
    }

    public String toString() {
```

```

        return id + " " + name;
    }
}

class Test {
    public static void main(String[] args) {

        TreeSet<Student> set = new TreeSet<>();

        set.add(new Student(1,"Z"));
        set.add(new Student(2,"A"));

        System.out.println(set);
    }
}

```

6. COMPARATOR

```

import java.util.*;

class Student {
    int id;
    String name;

    Student(int id,String name) {
        this.id = id;
        this.name = name;
    }

    public String toString() {
        return id + " " + name;
    }
}

class SortById implements Comparator<Student> {
    public int compare(Student a, Student b) {
        return a.id - b.id;
    }
}

```

7. HASHMAP

```
import java.util.*;

class Test {
    public static void main(String[] args) {

        HashMap<Integer,String> map = new HashMap<>();

        map.put(1,"A");
        map.put(2,"B");

        System.out.println(map.get(1));

        System.out.println(map.containsKey(2));

        for(Map.Entry<Integer,String> e : map.entrySet()) {
            System.out.println(e.getKey() + " " + e.getValue());
        }
    }
}
```

8. DEQUE

```
import java.util.*;

class Test {
    public static void main(String[] args) {

        Deque<Integer> dq = new ArrayDeque<>();

        dq.addFirst(10);
        dq.addLast(20);

        System.out.println(dq);

        dq.removeFirst();

        System.out.println(dq);
    }
}
```

9. JDBC

JDBC = Java Database Connectivity

Steps: 1. Import package 2. Load driver 3. Establish connection 4. Create statement 5. Execute query 6. Close connection

10. JDBC CONNECTION

```
import java.sql.*;

class Test {
    public static void main(String[] args) {

        try {

            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/test",
                "root",
                "root"
            );

            System.out.println("Connected");

            con.close();
        }
        catch(Exception e) {
            System.out.println(e);
        }
    }
}
```

11. INSERT USING JDBC

```
import java.sql.*;

class Test {
```

```

public static void main(String[] args) {

    try {
        Class.forName("com.mysql.cj.jdbc.Driver");

        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/test",
            "root",
            "root"
        );

        PreparedStatement ps = con.prepareStatement(
            "insert into student values(?,?)"
        );

        ps.setInt(1,1);
        ps.setString(2,"Aquib");

        ps.executeUpdate();

        System.out.println("Inserted");

        con.close();
    }
    catch(Exception e) {
        System.out.println(e);
    }
}
}

```

12. SELECT USING JDBC

```

import java.sql.*;

class Test {
    public static void main(String[] args) {

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/test",
                "root",
                "root"
            );

```

```

    );

    Statement st = con.createStatement();

    ResultSet rs = st.executeQuery("select * from student");

    while(rs.next()) {
        System.out.println(rs.getInt(1));
        System.out.println(rs.getString(2));
    }

    con.close();
}
catch(Exception e) {
    System.out.println(e);
}
}
}

```

MOST IMPORTANT DIFFERENCES

Topic	Difference
String vs StringBuilder	Immutable vs Mutable
Abstract class vs Interface	Partial abstraction vs Full abstraction
Method overloading vs overriding	Compile time vs Runtime
ArrayList vs LinkedList	Dynamic array vs Doubly linked list
HashSet vs TreeSet	Unordered vs Sorted
Comparable vs Comparator	Single sorting vs Multiple sorting
== vs equals()	Reference vs Content
throw vs throws	Manually throw vs Declare exception
Process vs Thread	Heavyweight vs Lightweight

TOP 50 VIVA QUESTIONS

1. Why Java is platform independent?
2. Difference between JDK, JRE, JVM?

3. Why main method is static?
4. What is constructor?
5. Difference between constructor and method?
6. What is method overloading?
7. What is method overriding?
8. What is inheritance?
9. What is polymorphism?
10. What is encapsulation?
11. What is abstraction?
12. Difference between abstract class and interface?
13. What is String pool?
14. Why String immutable?
15. Difference between String and StringBuilder?
16. What is static keyword?
17. What is final keyword?
18. What is this keyword?
19. What is super keyword?
20. What is exception?
21. Difference between checked and unchecked exception?
22. What is finally block?
23. What is throw?
24. What is throws?
25. What is custom exception?
26. What is multithreading?
27. Difference between process and thread?
28. Difference between Thread and Runnable?
29. What is synchronization?
30. What is deadlock?
31. What is lambda expression?
32. What is functional interface?
33. What is collection framework?
34. Difference between ArrayList and LinkedList?
35. Difference between HashSet and TreeSet?
36. What is HashMap?
37. Difference between Comparable and Comparator?
38. What is generic class?
39. What is serialization?
40. What is JDBC?
41. What are JDBC drivers?
42. What is PreparedStatement?
43. Difference between executeQuery and executeUpdate?
44. What is ResultSet?
45. What is package?
46. What is access modifier?
47. What is nested class?
48. What is anonymous class?
49. What is assertion?
50. Difference between compile-time and runtime polymorphism?

MOST IMPORTANT MCQ POINTS

1. Java supports multiple inheritance through interfaces.
 2. String is immutable.
 3. Object class is parent of all classes.
 4. JVM executes bytecode.
 5. JDK contains compiler.
 6. Constructor has no return type.
 7. finally block always executes.
 8. TreeSet stores sorted data.
 9. HashMap stores key-value pairs.
 10. Array index starts from 0.
 11. Interface methods are public abstract.
 12. static belongs to class.
 13. final variable cannot change.
 14. Thread.start() creates new thread.
 15. run() does not create new thread.
 16. Java does not support pointers directly.
 17. Java supports garbage collection.
 18. equals() compares contents.
 19. == compares references.
 20. Scanner belongs to java.util package.
-

NEOCOLAB CODING PREPARATION

Pattern 1 — Array Problem

```
import java.util.*;

class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int arr[] = new int[n];

        for(int i=0;i<n;i++) {
            arr[i] = sc.nextInt();
        }
    }
}
```

```
int sum = 0;

for(int i=0;i<n;i++) {
    sum += arr[i];
}

System.out.println(sum);
}
}
```

Pattern 2 — String Problem

```
import java.util.*;

class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        String s = sc.nextLine();

        String rev = "";

        for(int i=s.length()-1;i>=0;i--) {
            rev += s.charAt(i);
        }

        System.out.println(rev);
    }
}
```

Pattern 3 — OOP Problem

```
class Student {
    int id;
    String name;

    Student(int id,String name) {
        this.id = id;
        this.name = name;
    }
}
```

```

    }

    void display() {
        System.out.println(id + " " + name);
    }
}

class Main {
    public static void main(String[] args) {

        Student s = new Student(1,"Aquib");

        s.display();
    }
}

```

Pattern 4 — Collection Problem

```

import java.util.*;

class Main {
    public static void main(String[] args) {

        ArrayList<Integer> list = new ArrayList<>();

        list.add(10);
        list.add(20);
        list.add(30);

        Collections.sort(list);

        System.out.println(list);
    }
}

```

Pattern 5 — Exception Problem

```

class Main {
    public static void main(String[] args) {

```

```
try {  
    int a = 10/0;  
}  
catch(Exception e) {  
    System.out.println("Handled");  
}  
}
```

WRITTEN EXAM STRATEGY

1. First attempt easy questions.
 2. Write syntax correctly.
 3. Underline keywords.
 4. Draw diagrams for JVM/JDK/JRE.
 5. Write advantages and features pointwise.
 6. Always write examples.
 7. Write definitions in technical language.
 8. Practice previous year questions.
-

HOW TO SCORE FULL MARKS IN VIVA

1. Speak confidently.
2. Never stay silent.
3. Explain with examples.
4. Use keywords:
5. inheritance
6. polymorphism
7. abstraction
8. encapsulation

9. Know all syntax.
 10. Practice writing code on paper.
 11. Learn differences carefully.
-

LAST 7 DAYS REVISION PLAN

Day 1

- Unit I complete

Day 2

- Unit II complete

Day 3

- Unit III complete

Day 4

- Unit IV complete

Day 5

- Unit V complete

Day 6

- Unit VI complete

Day 7

- Full revision + coding practice + viva questions
-

FINAL CHECKLIST BEFORE EXAM

You must be able to write without seeing:

- Class and object program
- Constructor program
- Inheritance program
- Overriding program

- Interface program
- Exception handling program
- Multithreading program
- ArrayList program
- HashMap program
- TreeSet using Comparable
- JDBC connection program
- File handling program
- Lambda expression program
- Generic class program

If you can write all above from memory, you are exam ready.

GOLDEN RULES FOR 100% MARKS

1. Never memorize blindly.
 2. Understand syntax.
 3. Type code daily.
 4. Practice output prediction.
 5. Solve errors yourself.
 6. Revise frequently.
 7. Practice handwritten code.
 8. Focus on collections + JDBC + exception handling.
 9. Learn all important methods.
 10. Stay calm in coding exam.
-

ADDITIONAL IMPORTANT TOPICS THAT WERE MISSED EARLIER

1. IDENTIFIERS RULES

Identifiers are names of variables, methods, classes.

Rules:

- Cannot start with number
- Cannot use keywords
- Can contain letters, digits, _, \$
- Case sensitive

Valid:

```
int age;  
int _num;  
int $value;
```

Invalid:

```
int 1age;  
int class;
```

2. PACKAGE

Package is collection of related classes.

Benefits:

- Avoid naming conflict
- Access protection
- Reusability

Example:

```
package mypack;  
  
class Test {  
    public static void main(String[] args) {  
        System.out.println("Hello");  
    }  
}
```

Import package:

```
import java.util.Scanner;
```

3. GARBAGE COLLECTION

Unused objects are automatically destroyed.

Garbage collector belongs to JVM.

```
System.gc();
```

finalize() method:

```
protected void finalize() {  
    System.out.println("Destroyed");  
}
```

4. MEMORY ALLOCATION

Stack Memory

- Local variables
- Method calls

Heap Memory

- Objects
- Instance variables

5. PASS BY VALUE IN JAVA

Java is always pass by value.

```
class Test {  
  
    static void change(int x) {  
        x = 100;  
    }  
  
    public static void main(String[] args) {  
        int a = 10;  
        change(a);  
        System.out.println(a);  
    }  
}
```

Output:

10

6. CALL BY REFERENCE CONCEPT

Java does not support true call by reference. Object references are passed by value.

7. COMMANDS USED IN JAVA

Compile:

```
javac FileName.java
```

Run:

```
java FileName
```

8. BREAK STATEMENT

```
for(int i=1;i<=10;i++) {  
    if(i==5)  
        break;  
  
    System.out.println(i);  
}
```

9. CONTINUE STATEMENT

```
for(int i=1;i<=5;i++) {  
    if(i==3)  
        continue;  
}
```

```
System.out.println(i);  
}
```

10. LABELLED LOOP

```
outer:  
for(int i=1;i<=3;i++) {  
  
    for(int j=1;j<=3;j++) {  
  
        if(i==2)  
            break outer;  
  
        System.out.println(i + " " + j);  
    }  
}
```

11. INCREMENT AND DECREMENT

Pre Increment

```
++a
```

Post Increment

```
a++
```

12. ESCAPE SEQUENCES

Escape	Meaning
New line	
Tab	

Escape	Meaning
\	Backslash
\"	Double quote

13. INSTANCE INITIALIZER vs STATIC BLOCK

Static Block

Runs once when class loads.

```
static {  
    System.out.println("Static Block");  
}
```

Instance_INITIALIZER

Runs before constructor.

```
{  
    System.out.println("Instance Block");  
}
```

14. STATIC IMPORT

```
import static java.lang.Math.*;  
  
class Test {  
    public static void main(String[] args) {  
        System.out.println(sqrt(25));  
    }  
}
```

15. MATH CLASS IMPORTANT METHODS

```
Math.sqrt(25);  
Math.pow(2,3);  
Math.max(10,20);  
Math.min(10,20);  
Math.random();  
Math.abs(-5);  
Math.ceil(5.2);  
Math.floor(5.9);
```

16. CHARACTER CLASS METHODS

```
Character.isDigit('5');  
Character.isLetter('A');  
Character.isUpperCase('A');  
Character.isLowerCase('a');  
Character.toUpperCase('a');
```

17. IMPORTANT STRING METHODS

```
trim()  
indexOf()  
lastIndexOf()  
startsWith()  
endsWith()  
concat()  
isEmpty()  
compareTo()
```

18. STRINGBUFFER

Thread-safe mutable string.


```
StringBuffer sb = new StringBuffer("Hello");  
sb.append(" World");  
System.out.println(sb);
```

Difference:

- StringBuilder faster
- StringBuffer synchronized

19. ARRAY IMPORTANT POINTS

- Arrays are fixed size.
- Arrays store same datatype.
- Arrays are objects in Java.
- length is property.

```
arr.length
```

20. COPYING ARRAY

```
import java.util.Arrays;  
  
int a[] = {1,2,3};  
int b[] = Arrays.copyOf(a,3);
```

21. SORTING ARRAY

```
Arrays.sort(arr);
```

22. SEARCHING ARRAY

```
Arrays.binarySearch(arr,10);
```

23. ACCESS SPECIFIER WITH OVERRIDING

Access level cannot be more restrictive.

Valid:

```
protected -> public
```

Invalid:

```
public -> private
```

24. CONSTRUCTOR CHAINING

```
class Test {  
  
    Test() {  
        this(10);  
        System.out.println("Default");  
    }  
  
    Test(int x) {  
        System.out.println(x);  
    }  
}
```

25. SUPER() CONSTRUCTOR

```
class A {  
    A() {  
        System.out.println("A");  
    }  
}  
  
class B extends A {  
    B() {
```

```
        super();  
        System.out.println("B");  
    }  
}
```

26. DYNAMIC METHOD DISPATCH

```
class A {  
    void show() {  
        System.out.println("A");  
    }  
}  
  
class B extends A {  
    void show() {  
        System.out.println("B");  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        A obj = new B();  
        obj.show();  
    }  
}
```

Runtime decides method.

27. OBJECT CLONING

```
class Student implements Cloneable {  
    int id = 10;  
  
    public static void main(String[] args) throws Exception {  
  
        Student s1 = new Student();  
        Student s2 = (Student)s1.clone();  
  
        System.out.println(s2.id);  
    }  
}
```

```
}  
}
```

28. MARKER INTERFACE

Interface without methods.

Example:

- Serializable
- Cloneable

29. TYPES OF EXCEPTIONS

Checked Exception

Checked at compile time.

Examples:

- IOException
- SQLException

Unchecked Exception

Checked at runtime.

Examples:

- ArithmeticException
- NullPointerException
- ArrayIndexOutOfBoundsException

30. COMMON EXCEPTIONS

```
ArithmeticException  
NullPointerException  
NumberFormatException
```

```
ClassCastException  
IOException  
SQLException  
FileNotFoundException
```

31. EXCEPTION PROPAGATION

```
class Test {  
  
    static void check() {  
        int a = 10/0;  
    }  
  
    public static void main(String[] args) {  
  
        try {  
            check();  
        }  
        catch(Exception e) {  
            System.out.println(e);  
        }  
    }  
}
```

32. USER DEFINED PACKAGE

Compile:

```
javac -d . Test.java
```

Run:

```
java mypack.Test
```

33. FILE CLASS

```
import java.io.*;

class Test {
    public static void main(String[] args) {

        File f = new File("abc.txt");

        System.out.println(f.exists());
        System.out.println(f.getName());
        System.out.println(f.length());
    }
}
```

34. BUFFERED WRITER

```
import java.io.*;

class Test {
    public static void main(String[] args) throws Exception {

        BufferedWriter bw = new BufferedWriter(new FileWriter("a.txt"));

        bw.write("Hello");

        bw.close();
    }
}
```

35. BUFFERED READER FILE

```
import java.io.*;

class Test {
    public static void main(String[] args) throws Exception {

        BufferedReader br = new BufferedReader(new FileReader("a.txt"));
    }
}
```

```
String s;

while((s = br.readLine()) != null) {
    System.out.println(s);
}

br.close();
}
```

36. TRANSIENT KEYWORD

Transient variables are not serialized.

```
transient int password;
```

37. VOLATILE KEYWORD

Used in multithreading.

```
volatile boolean flag = true;
```

38. DAEMON THREAD

Background thread.

```
Thread t = new Thread();
t.setDaemon(true);
```

39. THREAD METHODS

```
sleep()
yield()
```

```
join()  
isActive()  
currentThread()
```

40. THREAD SLEEP

```
try {  
    Thread.sleep(1000);  
}  
catch(Exception e) {  
}
```

41. THREAD JOIN

```
t1.join();
```

Waits for thread completion.

42. COLLECTION HIERARCHY

Collection

- List
- ArrayList
- LinkedList
- Set
- HashSet
- TreeSet
- Queue
- PriorityQueue
- Deque

Map

- HashMap
 - TreeMap
-

43. ITERATOR

```
Iterator<Integer> it = list.iterator();

while(it.hasNext()) {
    System.out.println(it.next());
}
```

44. PRIORITY QUEUE

```
PriorityQueue<Integer> pq = new PriorityQueue<>();

pq.add(30);
pq.add(10);
pq.add(20);

System.out.println(pq.poll());
```

45. TREEMAP

```
TreeMap<Integer,String> map = new TreeMap<>();
```

Sorted according to keys.

46. VECTOR

```
Vector<Integer> v = new Vector<>();
```

Thread-safe dynamic array.

47. STACK CLASS

```
Stack<Integer> st = new Stack<>();

st.push(10);
st.push(20);

System.out.println(st.pop());
```

48. LEGACY CLASSES

- Vector
- Hashtable
- Stack

49. JDBC IMPORTANT INTERFACES

- Driver
- Connection
- Statement
- PreparedStatement
- ResultSet

50. STATEMENT vs PREPAREDSTATEMENT

Statement	PreparedStatement
Less secure	More secure
SQL injection possible	Prevents SQL injection
Slower	Faster

51. CRUD OPERATIONS

Operation	SQL
Create	INSERT
Read	SELECT
Update	UPDATE
Delete	DELETE

52. UPDATE USING JDBC

```
Statement st = con.createStatement();  
st.executeUpdate("update student set name='A' where id=1");
```

53. DELETE USING JDBC

```
Statement st = con.createStatement();  
st.executeUpdate("delete from student where id=1");
```

54. IMPORTANT JDBC METHODS

```
executeQuery()  
executeUpdate()  
next()  
getInt()  
getString()  
close()
```

55. SQL INJECTION

Hackers manipulate query using input.

PreparedStatement prevents it.

56. COMMON OUTPUT QUESTIONS

Always practice:

- Constructor output
 - Inheritance output
 - Static block output
 - finally block output
 - String comparison output
 - Thread output
 - Increment/decrement output
-

57. IMPORTANT PROGRAMS FOR EXAM

You must practice these programs without seeing notes:

1. Palindrome string
 2. Prime number
 3. Factorial
 4. Fibonacci
 5. Matrix addition
 6. Array sorting
 7. Employee class
 8. Bank account class
 9. Student management using ArrayList
 10. HashMap phonebook
 11. File copy program
 12. Exception handling demo
 13. Multithreading demo
 14. JDBC CRUD program
 15. Lambda sorting program
 16. Generic class
 17. Serialization program
 18. Comparable student sorting
 19. Comparator sorting by marks
 20. Interface implementation
-

58. VERY IMPORTANT THEORY QUESTIONS

1. Features of Java
 2. OOP principles
 3. Difference between C++ and Java
 4. Benefits of inheritance
 5. Benefits of multithreading
 6. Need of collection framework
 7. Importance of exception handling
 8. Advantages of JDBC
 9. Advantages of generics
 10. Difference between abstract class and interface
-

59. COMMON COMPILATION ERRORS

- Missing semicolon
 - Wrong class name
 - Wrong main method syntax
 - Variable not initialized
 - Missing import
 - Wrong braces
 - Incompatible types
-

60. MAIN METHOD SIGNATURE

```
public static void main(String[] args)
```

Alternative valid form:

```
public static void main(String args[])
```

61. IMPORTANT PACKAGES

Package	Purpose
java.lang	Core classes
java.util	Collections, Scanner

Package	Purpose
java.io	File handling
java.sql	JDBC
java.time	Date and time

62. JAVA NAMING CONVENTIONS

Element	Style
Class	PascalCase
Variable	camelCase
Method	camelCase
Constant	UPPER_CASE

Example:

```
class StudentData
int totalMarks
final double PI
```

63. IMPORTANT COLLECTION METHODS

```
add()
remove()
contains()
clear()
size()
isEmpty()
```

64. COLLECTIONS UTILITY CLASS

```
Collections.sort(list);
Collections.reverse(list);
```

```
Collections.max(list);  
Collections.min(list);
```

65. COMPARING STRINGS

```
String a = "Hello";  
String b = "Hello";  
  
System.out.println(a == b);  
System.out.println(a.equals(b));
```

66. IMMUTABLE CLASS

String class is immutable.

Benefits:

- Security
 - Thread safety
 - Performance
-

67. AUTOBOXING AND UNBOXING

```
Integer x = 10;  
int y = x;
```

68. TYPE CONVERSION HIERARCHY

```
byte -> short -> int -> long -> float -> double
```

69. IMPORTANT JAVA FILE EXTENSIONS

File	Extension
Java source	.java
Bytecode	.class

70. BYTECODE

Bytecode is generated after compilation. Executed by JVM.

FINAL RECHECK OF SYLLABUS COVERAGE

Covered:

✓ Introduction to Java ✓ Java structure ✓ JDK JRE JVM ✓ Primitive datatypes ✓ Operators ✓ Loops ✓ Arrays ✓ Varargs ✓ Enum ✓ OOP concepts ✓ Constructors ✓ Overloading ✓ this keyword ✓ String and StringBuilder ✓ Inheritance ✓ Polymorphism ✓ Overriding ✓ super keyword ✓ final keyword ✓ Object class ✓ toString and equals ✓ instanceof ✓ Abstract class ✓ Interface ✓ Static/default methods ✓ Nested classes ✓ Lambda expressions ✓ Dates ✓ Exceptions ✓ Assertions ✓ File handling ✓ Serialization ✓ Generics ✓ Threads ✓ Synchronization ✓ Collections ✓ ArrayList ✓ TreeSet ✓ Comparable ✓ Comparator ✓ HashMap ✓ Deque ✓ JDBC ✓ CRUD operations ✓ Viva questions ✓ MCQ preparation ✓ Coding patterns ✓ Practical programs ✓ Exam strategy

71. ACCESSING COMMAND LINE ARGUMENT LENGTH

```
System.out.println(args.length);
```

72. PARSING STRING TO INTEGER

```
int x = Integer.parseInt("100");
```


73. FORMATTED OUTPUT

```
System.out.printf("%d %s",10,"Java");
```

74. METHODS OF OBJECT CLASS

```
toString()  
equals()  
hashCode()  
getClass()  
clone()  
finalize()
```

75. SHALLOW COPY vs DEEP COPY

Shallow Copy	Deep Copy
Copies reference	Copies complete object
Faster	Slower

76. METHOD SIGNATURE

Method signature includes:

- Method name
- Parameter list

Return type is not part of method signature.

77. RULES OF METHOD OVERLOADING

Overloading can change:

- Number of parameters

- Datatype of parameters
- Order of parameters

Cannot overload only by changing return type.

78. COVARIANT RETURN TYPE

```
class A {  
    A show() {  
        return this;  
    }  
}  
  
class B extends A {  
    B show() {  
        return this;  
    }  
}
```

79. FINALIZE vs FINALLY vs FINAL

Keyword	Purpose
final	Constant/restriction
finally	Exception block
finalize()	Garbage collection

80. INTERFACE INHERITANCE

```
interface A {  
    void show();  
}  
  
interface B extends A {  
    void display();  
}
```

81. MULTIPLE INHERITANCE USING INTERFACE

```
interface A {  
    void show();  
}  
  
interface B {  
    void display();  
}  
  
class Test implements A,B {  
  
    public void show() {  
        System.out.println("Show");  
    }  
  
    public void display() {  
        System.out.println("Display");  
    }  
}
```

82. LOCAL INNER CLASS

```
class Outer {  
  
    void test() {  
  
        class Inner {  
            void show() {  
                System.out.println("Local Inner");  
            }  
        }  
  
        Inner i = new Inner();  
        i.show();  
    }  
}
```

83. LAMBDA WITH PARAMETERS

```
interface Add {
    int sum(int a,int b);
}

class Test {
    public static void main(String[] args) {

        Add obj = (a,b) -> a+b;

        System.out.println(obj.sum(10,20));
    }
}
```

84. METHOD REFERENCE

```
class Test {

    static void display() {
        System.out.println("Hello");
    }

    public static void main(String[] args) {

        Runnable r = Test::display;

        r.run();
    }
}
```

85. AUTO CLOSEABLE

Classes used in try-with-resources must implement:

```
AutoCloseable
```

86. DIFFERENCE BETWEEN ERROR AND EXCEPTION

Error	Exception
Serious problem	Can be handled
Example: OutOfMemoryError	Example: IOException

87. THROWING CUSTOM EXCEPTION

```
class InvalidAgeException extends Exception {  
  
    InvalidAgeException(String msg) {  
        super(msg);  
    }  
}  
  
class Test {  
  
    static void check(int age) throws InvalidAgeException {  
  
        if(age < 18)  
            throw new InvalidAgeException("Not eligible");  
    }  
}
```

88. RANDOM CLASS

```
import java.util.*;  
  
Random r = new Random();  
  
System.out.println(r.nextInt(100));
```

89. SCANNER IMPORTANT METHODS

```
nextInt()  
nextDouble()  
next()  
nextLine()  
nextBoolean()
```

90. DIFFERENCE BETWEEN next() AND nextLine()

next()	nextLine()
Reads one word	Reads full line

91. FILE METHODS

```
createNewFile()  
delete()  
mkdir()  
list()  
canRead()  
canWrite()
```

92. PRINTWRITER

```
import java.io.*;  
  
PrintWriter pw = new PrintWriter("a.txt");  
  
pw.println("Hello");  
  
pw.close();
```

93. BYTE STREAMS vs CHARACTER STREAMS

Byte Streams	Character Streams
Binary data	Text data
FileInputStream	FileReader

94. FILEINPUTSTREAM

```
FileInputStream fis = new FileInputStream("a.txt");
```

95. FILEOUTPUTSTREAM

```
FileOutputStream fos = new FileOutputStream("a.txt");
```

96. DESERIALIZATION

```
FileInputStream fis = new FileInputStream("a.txt");
ObjectInputStream ois = new ObjectInputStream(fis);

Student s = (Student)ois.readObject();
```

97. GENERIC METHOD

```
class Test {

    <T> void display(T data) {
        System.out.println(data);
    }
}
```

98. UPPER BOUNDED WILDCARD

```
List<? extends Number> list;
```

99. LOWER BOUNDED WILDCARD

```
List<? super Integer> list;
```

100. THREAD LIFE CYCLE

States:

- New
 - Runnable
 - Running
 - Waiting
 - Dead
-

101. SYNCHRONIZED BLOCK

```
synchronized(this) {  
    System.out.println("Critical section");  
}
```

102. DEADLOCK

Two threads waiting forever for each other.

103. RACE CONDITION

Multiple threads access shared data simultaneously causing incorrect output.

104. CONCURRENTMODIFICATIONEXCEPTION

Occurs when collection modified during iteration.

105. MAP IMPORTANT METHODS

```
put()  
get()  
remove()  
containsKey()  
containsValue()  
keySet()  
values()  
entrySet()
```

106. HASHMAP FEATURES

- Allows one null key
 - Allows multiple null values
 - Unordered
 - Not synchronized
-

107. TREEMAP FEATURES

- Sorted by keys
 - No null key
-

108. SET IMPORTANT POINTS

- No duplicates
 - HashSet unordered
 - TreeSet sorted
-

109. LIST IMPORTANT POINTS

- Allows duplicates
 - Maintains insertion order
-

110. QUEUE IMPORTANT METHODS

```
add()  
offer()  
poll()  
peek()  
remove()
```

111. JDBC DRIVER TYPES

1. Type 1 — JDBC ODBC Bridge
2. Type 2 — Native API
3. Type 3 — Network Protocol
4. Type 4 — Thin Driver

Most used:

- Type 4
-

112. RESULTSET METHODS

```
next()  
previous()  
first()  
last()  
absolute()
```

113. executeQuery() vs executeUpdate()

executeQuery()	executeUpdate()
Used for SELECT	Used for INSERT UPDATE DELETE
Returns ResultSet	Returns int

114. BATCH PROCESSING IN JDBC

```
Statement st = con.createStatement();

st.addBatch("insert into student values(1,'A')");
st.addBatch("insert into student values(2,'B')");

st.executeBatch();
```

115. DRIVERMANAGER

Used to establish database connection.

```
DriverManager.getConnection()
```

116. COMMON RUNTIME ERRORS

- Divide by zero
- Null pointer access
- Array index out of bounds
- Infinite loop
- Wrong type casting

117. IMPORTANT ASCII VALUES

Character	ASCII
A	65
a	97
0	48

118. TYPE OF POLYMORPHISM

1. Compile-time polymorphism
2. Method overloading
3. Runtime polymorphism
4. Method overriding

119. ENCAPSULATION

Binding data and methods together.

Use private variables with getters/setters.

```
class Student {  
  
    private int id;  
  
    void setId(int id) {  
        this.id = id;  
    }  
  
    int getId() {  
        return id;  
    }  
}
```

120. ABSTRACTION

Showing essential details and hiding implementation.

Achieved using:

- Abstract class
 - Interface
-

121. PACKAGES

Package is a group of related classes and interfaces.

Example:

```
package mypack;

class Test {
    public static void main(String[] args) {
        System.out.println("Package Example");
    }
}
```

Importing package:

```
import java.util.*;
```

Common packages:

- java.lang
 - java.util
 - java.io
 - java.sql
 - java.time
-

122. STATIC BLOCK

```
class Test {
```

```
static {
    System.out.println("Static Block Executed");
}

public static void main(String[] args) {
    System.out.println("Main Method");
}
}
```

Static block executes before main method.

123. GARBAGE COLLECTION

Garbage collector destroys unused objects automatically.

```
System.gc();
```

124. finalize() METHOD

```
class Test {

    protected void finalize() {
        System.out.println("Object destroyed");
    }

}
```

Called before garbage collection.

125. MEMORY AREAS IN JAVA

Area	Purpose
Heap	Objects
Stack	Method calls/local variables
Method Area	Class metadata

126. break STATEMENT

```
for(int i=1;i<=10;i++) {  
  
    if(i==5)  
        break;  
  
    System.out.println(i);  
}
```

127. continue STATEMENT

```
for(int i=1;i<=5;i++) {  
  
    if(i==3)  
        continue;  
  
    System.out.println(i);  
}
```

128. LABELED LOOP

```
outer:  
for(int i=1;i<=3;i++) {  
  
    for(int j=1;j<=3;j++) {  
  
        if(i==2)  
            break outer;  
    }  
}
```

129. STRINGBUFFER

Mutable and thread-safe.

```
StringBuffer sb = new StringBuffer("Java");

sb.append(" Programming");

System.out.println(sb);
```

Difference:

StringBuilder	StringBuffer
Faster	Slower
Not synchronized	Synchronized

130. CHARACTER CLASS

```
Character.isDigit('1');
Character.isLetter('A');
Character.isUpperCase('A');
Character.toLowerCase('A');
```

131. MATH CLASS

```
Math.sqrt(25);
Math.pow(2,3);
Math.max(10,20);
Math.min(10,20);
Math.random();
Math.abs(-10);
```

132. CONSTRUCTOR CHAINING

```
class Test {

    Test() {
        this(10);
    }
}
```



```
        System.out.println("Default");
    }

    Test(int x) {
        System.out.println(x);
    }
}
```

133. DYNAMIC METHOD DISPATCH

```
class A {
    void show() {
        System.out.println("A");
    }
}

class B extends A {
    void show() {
        System.out.println("B");
    }
}

class Test {
    public static void main(String[] args) {

        A obj = new B();

        obj.show();
    }
}
```

Runtime decides which method executes.

134. CLONING

```
class Student implements Cloneable {

    int id = 1;

    public static void main(String[] args) throws Exception {
```

```
Student s1 = new Student();

Student s2 = (Student)s1.clone();

System.out.println(s2.id);
    }
}
```

135. MARKER INTERFACE

Interface without methods.

Examples:

- Serializable
- Cloneable

136. CHECKED vs UNCHECKED EXCEPTION

Checked	Unchecked
Checked at compile time	Checked at runtime
IOException	ArithmeticException

137. BUFFEREDWRITER

```
import java.io.*;

BufferedWriter bw = new BufferedWriter(new FileWriter("a.txt"));

bw.write("Hello");

bw.close();
```

138. transient KEYWORD

transient variable is not serialized.

```
transient int password;
```

139. volatile KEYWORD

Used in multithreading.

Value updated directly from main memory.

```
volatile boolean flag = true;
```

140. DAEMON THREAD

Runs in background.

```
Thread t = new Thread();  
  
t.setDaemon(true);
```

141. ITERATOR

```
Iterator<Integer> it = list.iterator();  
  
while(it.hasNext()) {  
    System.out.println(it.next());  
}
```

142. PRIORITYQUEUE

```
PriorityQueue<Integer> pq = new PriorityQueue<>();

pq.add(30);
pq.add(10);
pq.add(20);

System.out.println(pq.poll());
```

143. VECTOR

```
Vector<Integer> v = new Vector<>();

v.add(10);
v.add(20);
```

Vector is synchronized.

144. STACK CLASS

```
Stack<Integer> st = new Stack<>();

st.push(10);
st.push(20);

System.out.println(st.pop());
```

145. CRUD OPERATIONS

Operation	SQL
Create	INSERT
Read	SELECT
Update	UPDATE

Operation	SQL
Delete	DELETE

146. JAVA NAMING CONVENTIONS

Item	Convention
Class	PascalCase
Method	camelCase
Variable	camelCase
Constant	UPPER_CASE

Example:

```
class StudentData {
    int studentId;

    final double PI_VALUE = 3.14;
}
```

147. PASS BY VALUE IN JAVA

Java is always pass by value.

148. IMPORTANT STRING PROGRAMS

Palindrome String

```
import java.util.*;

class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
```

```

String s = sc.nextLine();

String rev = "";

for(int i=s.length()-1;i>=0;i--) {
    rev += s.charAt(i);
}

if(s.equals(rev))
    System.out.println("Palindrome");
else
    System.out.println("Not Palindrome");
}
}

```

149. IMPORTANT ARRAY PROGRAM

Find Largest Element

```

import java.util.*;

class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int arr[] = new int[n];

        for(int i=0;i<n;i++) {
            arr[i] = sc.nextInt();
        }

        int max = arr[0];

        for(int i=1;i<n;i++) {
            if(arr[i] > max)
                max = arr[i];
        }

        System.out.println(max);
    }
}

```

```
}  
}
```

150. IMPORTANT COLLECTION PROGRAM

Frequency Count Using HashMap

```
import java.util.*;  
  
class Main {  
    public static void main(String[] args) {  
  
        int arr[] = {1,2,2,3,1,2};  
  
        HashMap<Integer,Integer> map = new HashMap<>();  
  
        for(int x : arr) {  
  
            if(map.containsKey(x)) {  
                map.put(x,map.get(x)+1);  
            }  
            else {  
                map.put(x,1);  
            }  
        }  
  
        System.out.println(map);  
    }  
}
```

FINAL FINAL RECHECK

Additional topics now covered:

✓ Identifiers ✓ Packages ✓ Garbage collection ✓ Memory allocation ✓ break continue labels ✓ Static block ✓ Math class ✓ Character class ✓ StringBuffer ✓ Constructor chaining ✓ Dynamic method dispatch ✓ Cloning ✓ Marker interface ✓ Checked and unchecked exceptions ✓ File class ✓ BufferedWriter ✓ Transient and volatile ✓ Daemon thread ✓ Iterator ✓ PriorityQueue ✓ TreeMap ✓ Vector ✓ Stack ✓ CRUD operations ✓ Naming conventions ✓ finalize/final/finally ✓ Interface inheritance ✓ Local inner class ✓ Method reference ✓ Byte

streams ✓ Deserialization ✓ Generic methods ✓ Thread life cycle ✓ Deadlock ✓ Race condition ✓ JDBC driver types ✓ Encapsulation ✓ Abstraction

END

If you master every topic and code written in these notes and practice them repeatedly, you will be fully prepared for:

- Theory
- MCQs
- Viva
- NeoColab coding test
- Practical examination
- End-term examination

Master these notes completely and you will have strong command over CSE310 Programming in Java.